

What Breaks for VRAMlets

Ten failures that happened on the compressed path, not merely near it.

The goal of aggressive compression is simple: fit more model, context, or users into fixed hardware. Here **super-sparse** is an umbrella for low-bit weights or cache state, ternary or pruned representations, structured sparsity, and the converters and kernels needed to run them.

This edition keeps a case only when a matched control, an enable/disable comparison, or a repaired compressed path ties the optimization to measurable harm. Ordinary prompt, routing, loading, and serving bugs are out.

What survived the filter

1. **BitsAndBytes serving produced garbage.** A four-bit Llama-3 path answered normally through Transformers and emitted repetition and nonsense through vLLM's then-new BnB implementation.
2. **A quantized KV-cache bug produced gibberish.** The DeepSeek-V4 cache path omitted a required rotation; repairing the cache restored normal output without changing the weights.
3. **Four-bit weights multiplied tool-name errors.** The headline score stayed nearly flat while tool-name hallucinations rose by up to **2.5×** in the tested agents.
4. **INT4 and INT3 inflated reasoning traces.** Some tested models kept their final-answer accuracy while producing longer, more repetitive work.
5. **Dynamic FP8 erased its own memory saving.** A runtime retained both allocations: about **16 GiB** with the faulty path versus **10 GiB** when its memory-pool interaction was bypassed.
6. **Quantized CPU offloading ignored the GPU memory budget.** A BitsAndBytes INT8 load exhausted a 7.62 GiB GPU despite automatic CPU offloading and an explicit 3 GiB device budget; removing the quantization configuration avoided the CUDA OOM.
7. **A 2:4-plus-W4A16 kernel corrupted output.** The sparse stage answered normally; the combined quantized path failed until the `marlin_24_cuda_kernel` defect was identified.
8. **One 2:4 sparse operation took 2.7× as long as dense.** On the reported H100 shape, sparse took **0.657 ms** and dense took **0.242 ms**.
9. **At two bits, the answer signal collapsed inside tested models.** Four-bit failures looked like accumulated distortion; tested two-bit factual-recall failures lost the internal computation needed to recover the answer.
10. **A bad Q4 conversion quantized the wrong attention tensor.** The upload script put MXFP4 into unintended tensors, including an attention-gate tensor; the maintainer replaced the files.

The reusable test

Hold the model, prompt, task, hardware, and sampling fixed. Change only the low-bit or sparse path. Count semantic errors, completed jobs, generated tokens, latency, peak memory, and concurrency. If the failure does not follow that intervention or disappear when it is repaired, it does not belong in this report.

Ten errors caused by super-sparse paths

Every case has three parts: the compression intervention, an observable regression, and a control or repair that ties the regression to that path. **Direct** means the compressed path was compared with a matched alternative. **Mechanism-confirmed** means the source identified the broken representation, converter, or kernel.

1. A BitsAndBytes serving path turned a normal four-bit model into garbage

The reason to use BitsAndBytes was memory: the reported vLLM load put Llama-3-8B-Instruct into about **5.3 GB** of GPU weight memory. The [vLLM #5569](#) reproduction then asked "Hi!" and received the followed by exclamation marks until the **128-token** limit. An eager-mode banana-bread request produced a different stream of repeated letters and fragments.

The control made this a quantized-path failure rather than a vague bad-model story. The reporter loaded the same official model with Transformers and `load_in_4bit=True`; it returned a normal greeting. A maintainer reproduced the vLLM failure, found that the BitsAndBytes path failed with CUDA graphs, and still observed gibberish in eager mode. The defect therefore belonged to that then-new serving implementation, not to Llama-3 or four-bit inference in general.

Evidence grade: direct compressed-path failure. The source isolates the failing integration but does not identify one final arithmetic defect. The scope is vLLM 0.5.0.post1, the reported model and hardware paths, and that implementation state.

2. A quantized KV-cache bug turned normal output into gibberish

A transformer keeps information about previous tokens in a **key-value cache**, or KV cache. Quantizing it saves memory as prompts and concurrent users grow. In the [DeepSeek-V4 llama.cpp repair](#), the compressed cache omitted a required Hadamard rotation. Attention then read numerically valid cache values in the wrong representation.

The error signature was binary: the Q4 KV-cache path produced gibberish before the patch and normal text after it. The model weights, prompt, and cache setting stayed fixed; only the cache implementation changed. That is the clean causal receipt: **quantized cache before repair → nonsense; repaired quantized cache → intelligible output**.

A separate [llama.cpp report](#) supplies weaker field support. Long Q8/Q4-cache conversations could suddenly become stuck in gibberish, while restarting or disabling KV-cache quantization restored normal output. It did not isolate the final defect, so it corroborates rather than defines this finding.

Evidence grade: mechanism-confirmed. This proves one DeepSeek-V4/llama.cpp cache-representation bug. It does not mean all Q4 KV caches are unsafe.

3. Four-bit weights produced up to 2.5× more tool-name errors

A compressed agent can keep its benchmark score while becoming less trustworthy. [Flat Score, Amplified Failures](#) compared post-training **four-bit weights** with higher-precision agents while keeping activations and working state at 16 bits. Across the tested model families and domains, the standard score looked nearly unchanged under the paper's equivalence test.

The error channel did change. Tool-name hallucinations rose by up to **2.5×** in volume. Retail entity errors moved in the same direction. The benchmark appeared stable because its tolerance absorbed the additional failures; the low-bit agents were not behaviorally equivalent inside the task.

The causal contrast is controlled weight precision with the rest of the execution setup held fixed. The failure is not "quantization lowers every score." It is sharper: **four-bit weights increased specific decision errors that the aggregate score hid.**

Evidence grade: direct controlled failure. The result is scoped to the tested agents, tasks, and PTQ setup. It does not establish that every action boundary requires high precision.

4. Low-bit reasoning used more tokens without improving answers

Final-answer accuracy can conceal extra failed work. [Quantization Inflates Reasoning](#) compared low-bit versions of the same model families across math, code, science, and agentic tool-use tasks. Some INT4 and INT3 models retained much of their final accuracy while generating longer and more repetitive traces.

That is an operational error even when the last answer survives. Extra tokens consume decode time and serving capacity. Repetition also signals that the model spent more steps recovering, retrying, or circling around a damaged trajectory. A score that checks only the final string omits that cost.

The causal variable was bit width. The error signature was **more generated work per completed answer**, not necessarily a lower top-line score. A deployment test should therefore count completed jobs, generated tokens, malformed actions, retries, wall time, and energy together.

Evidence grade: direct controlled failure. The magnitude depends on model and task. The paper does not say that every low-bit trace grows or that every extra token is wrong.

5. Dynamic FP8 retained both weight copies and erased the memory saving

Dynamic FP8 should replace a larger weight representation with a smaller runtime one. In [vLLM #19855](#), the loader instead interacted badly with sleep mode and the CUDA memory pool. The runtime retained allocations that should not have coexisted.

The control was unusually clear: no quantization used about **16 GiB**, the faulty dynamic-FP8 path also used about **16 GiB**, and dynamic FP8 with the memory-pool interaction bypassed used about **10 GiB**. The quantized path delivered no saving until that allocator interaction was removed. A maintainer confirmed the limitation and later pointed to a fix.

Evidence grade: direct runtime failure. The promised capacity, not model quality, failed. This demonstrates one dynamic-load and sleep-mode interaction; it does not imply that a prequantized FP8 checkpoint has the same peak-memory behavior.

6. Quantized CPU offloading ignored the GPU memory budget

BitsAndBytes INT8 should make a large model smaller on GPU, while `llm_int8_enable_fp32_cpu_offload=True` leaves overflow weights on the CPU. In [Transformers #43873](#), a 22B model instead exhausted a laptop GPU with **7.62 GiB** of VRAM. The load failed while trying to allocate another **36 MiB**.

The reporter used `device_map="auto"`, `low_cpu_mem_usage=True`, and then tested an explicit `max_memory={0: "3GiB", "cpu": "64GiB"}` budget. The explicit budget did not prevent the GPU OOM. The matched control was counterintuitive but decisive: removing `quantization_config=bnb_config` did not produce a CUDA OOM, although execution was slow as expected for CPU offloading.

The issue therefore demonstrates a quantization-plus-offload placement failure under the reported stack, not a generic claim that the uncompressed 22B model fits in GPU memory. The source does not yet establish whether Transformers, Accelerate, or the BitsAndBytes integration owns the final defect.

Evidence grade: direct compressed-path failure; mechanism unresolved. The scope is Transformers 4.57.3, Accelerate 1.12.0, BitsAndBytes 0.49.1, `ejschwartz/decaf-v1-22b`, and the reported RTX 4070 Laptop GPU. The issue remained open when this addendum was written.

7. A semi-structured W4A16 kernel corrupted an otherwise normal sparse model

The [vLLM #10819](#) recipe combined **50% 2:4 sparsity** with four-bit GPTQ weights and 16-bit activations for Qwen2.5-7B. The compressed model's output became abnormal during vLLM inference.

The reporter supplied the decisive separation. The sparse-stage model, before the added W4A16 path, answered normally. The combined quantized stage failed. The reporter later identified `marlin_24_cuda_kernel` as the cause and linked the repair. This rules out the initial explanation that pruning alone simply destroyed model quality.

The error signature was therefore **normal sparse-stage output → corrupted output through the combined 2:4/W4A16 Marlin path**. It was an execution-kernel defect in the super-sparse stack, not an unavoidable quality cost of deleting half the weights.

Evidence grade: mechanism-confirmed. The source isolates the combined kernel path. It does not establish whether plain 2:4 sparsity or plain W4A16 would fail independently in every environment.

8. A sparse H100 operation took 2.7× as long as dense

Sparsity removes arithmetic on paper; it does not guarantee a faster runtime. [PyTorch #153825](#) compared a valid semi-structured **2:4** sparse operation with its dense counterpart on an H100. For the reported **3072 × 10240** shape, sparse took about **0.657 ms** and dense took about **0.242 ms**.

The sparse operation therefore took roughly **2.7× as long**. The report also traced substantial time to matrix-description setup, algorithm selection, and plan initialization. The representation reduced nominal work while the available software path added more overhead than it removed.

This is a physical failure, not a model-quality failure. The optimization promised speed and delivered a slowdown under that measured path. The correct comparison is end-to-end latency at the real shape, dtype, phase, and reuse pattern, not FLOP count alone.

Evidence grade: direct benchmark failure. The number belongs to one H100 setup and shape. It is not a universal sparse-to-dense ratio.

9. Two-bit factual recall collapsed inside the tested models

[From Signal Degradation to Computation Collapse](#) used controlled bit-width sweeps on tested GPTQ models and factual-recall tasks. The authors found two qualitatively different failure modes rather than one smooth quality curve.

At four bits, the correct-token signal could still appear in intermediate layers and then be damaged by accumulated error. At two bits, the tested failures were deeper: the correct signal did not form, clean-activation repair failed, and low-rank compensation did not recover it. The representation had crossed from noisy computation into **computation collapse** for those cases.

The error signature is internal, but operationally important. A decoder cannot recover an answer that the model never constructs. Once this transition occurs, prompt tricks or output cleanup attack the wrong layer of the problem.

Evidence grade: direct controlled failure. The boundary depends on model, method, and task. The paper does not prove that “two bits always collapse” or “four bits are always safe.”

10. A bad Q4 conversion quantized the wrong attention tensor

A label such as “Q4” does not specify which tensors received which low-bit format. In the [Qwen3.5-35B-A3B-GGUF correction](#), the uploaded file’s conversion script inserted MXFP4 into unintended tensors, including an attention-gate tensor. Linked perplexity and KLD checks showed a discrepancy that should not have been attributed to the Q4 family name alone.

The maintainer acknowledged the script problem, replaced the upload, and supplied corrected temporary files. The causal chain was concrete: **converter selection error → wrong tensor format → measurable quality discrepancy → corrected artifact**.

This is a super-sparse error because the damage came from how the low-bit representation was assigned, not because an unrelated server happened to load a quantized file. It also explains why two files with the same coarse label can behave differently.

Evidence grade: mechanism-confirmed conversion failure. The incident proves one artifact defect. It does not prove that Q4_K_XL or MXFP4 is intrinsically bad.

Corpus boundary

The 1,640-record complaint corpus and its aggregate figures remain frozen at the **2026-08-08** sweep boundary. Finding 6 is a post-sweep addendum discovered while auditing that sweep’s recall. It is included in this causal report but is not included in the published corpus totals.